Universidad Autónoma de Baja California

FACULTAD DE CIENCIAS QUÍMICAS E INGENIERÍA

Organización y Arquitectura de las Computadoras.

Práctica No. 3:

multiplicación, división y números.

Alumno:

Joshua Osorio O. – 1293271

Docente:

José Isabel García R.

07/03/2026

Tabla de contenido

Objetivo 2
Desarrollo 2
Conclusiones 6
Dificultades 6
Bibliografía 7

Objetivo

Seleccionar el modo de direccionamiento adecuado, para manejo de memoria en aplicaciones de sistemas basados en microprocesador, mediante el análisis de los diversos modos de direccionamiento existentes, de forma responsable y eficiente.

Desarrollo

1. Capturar dos números y guardarlos en sus respectivas variables en memoria no inicializada.

```
section .bss                        ;Datos no inicializados
    num1        resb    1           ; 1 byte va desde 0 hasta 255.
    num2        resb    1
_start:
    call    clrscr                  ; Limpiamos pantalla
    call    salto
    mov     edx,        msj1        ; Mostrar mensaje 1
    call    puts
    call    getche                  ; Capturar numero 1
    mov     ebx,        num1
    sub     al,         48          ; Realizar casteo a numero.
    mov     [ebx],      al          ; Guardar numero en variable.
    mov     al,         'x'         ; Mostrar mensaje 2
    call    putchar
    call    getche                  ; Capturar numero 2
    mov     edx,        num2
    sub     al,         48          ; Realizar casteo a numero.
    mov     [edx],      al
    mov     al,         '='         ; Mostrar mensaje 2
    call    putchar
    ;-------------- Fin --------------
    mov     eax,        1           ;Carga la instruccion de salida de programa.
    mov     ebx,        0           ;Indica que termino correctamente, como un return 0 en c.
    int     80h                     ;Llamada a kener con las anteriores mensajes. Fin del pro-
grama main.
```

2. Implementar un procedimiento que por medio de sumas consecutivas utilizando la instrucción loop explicada en clase genere el mismo funcionamiento que una multiplicación.
    a. Ejemplo: 3x4=12
        i. 4+4+4=12.
        ii. El primer número indica el número de veces a sumar el segundo número.

```
    ;-------------- 1 - Para multiplicar --------------
    mov     edx,        num1
    mov     cl,         [edx]
    mov     edx,        num2
    mov     bl,         [edx]
    mov     eax,        0           ; Limpiar el registro completo para multi y printHex
    call    multi                   ; Como el resultado lo regresa en al ya no es necesario car-
gar en eax solo limpiarlo.
    mov     esi,        cad         ; Es necesario para utilizar printHex
    call    printHex
    call    salto
```

Se creo una subrutina para multiplicar

```
multi:
    ; Entrada:  cl  ->  Veces que se repite la suma.
    ;           bl  ->  numero a sumar
    ; Salida:   al  ->  Resultado
    mov     al, 0           ;Limpiar todo el registro eax porque ahi guardamos el resultado.
    .ciclo:
        add     al,bl       ; Multiplicar es sumar tantas veces el mismo numero.
        loop .ciclo         ; Esta funcion revisa el registo ecx y en automatico decrementa. Solo
decrementa ebx
    ret
```

3. Implementar un procedimiento que por medio de restas consecutivas utilizando la instrucción jmp explicada en clase genere el mismo funcionamiento que una división de enteros.
    a. Ejemplo: 12/3=4

i. 12-3=9; 9-3=6; 6-3=3; 3-3=0.
ii. El primer número es el resultado de la anterior función, el segundo número es el que se le restara al primero hasta dar con el resultado de 0.

Anteriormente se utilizo la subrutina de multiplizar donde guardo el resultado en al, por eso en la segunda linea se mueve el resultdo guardado en el registro al se copio a bl

```
    ;-------------- 2 - Para Dividir --------------
    mov     bl,         al          ; Muevo el resultado para utiliarza la funcion dividir.
    mov     edx,        msj2        ; Mostrar mensaje
    call    puts
    call    printHex                ; Mostrar resultado anterior guardado en al
    mov     al,        '/'
    call    putchar
    mov     al,         [num2]
    call    printHex
    mov     al,         '='
    call    putchar
    mov     edx,        num2
    mov     eax,        0
    mov     cl,         [edx]
    mov     eax,        0
    call    divi
    call    printHex
    call    salto
```

```
divi:
    ;   Entrada:  cl -> Divisor
    ;             bl -> Dividendo
    ;   Salida:   al-> Resultado
    mov al, 0               ; cociente = 0
    .bucle:
        cmp bl, cl          ; Dividendo es menor que divisor
        jb .fin_divi        ; Si es menor, terminamos
        sub bl, cl          ; Dividendo -= divisor
        inc al              ; cociente++
        jmp .bucle
    .fin_divi:
    ret
```

4. Implementar un procedimiento que muestre los números desde el 0 al 100 utilizando la subrutina printHex que se les entregó en moodle, únicamente utilizando las instrucciones vistas hasta el momento.
    a. Ejemplo: 0, 1, 2, 3, … , 100

```
    ; ------------- 3 - Para numeros 0 al 100 -------------
    mov     edx,        msj3        ; Mostrar mensaje
    call    puts
    mov     dh,         100
    mov     cl,         1
    mov     dl,         0
    mov     eax,        0
    call numeros
    call salto
```

```
numeros:
    ;   Entrada:    dl -> Inicio
    ;               cl -> Incrementos
    ;               dh -> Fin
    ;   Utiliza:    eax
    ;   Salida:     Muesta los valores en consola
    .bucle:
        cmp     dh, dl
        jb      .fin_divi       ; Si es menor, terminamos
        mov     al,         dl
        call    printHex
        call    salto
        add     dl, cl
        jmp .bucle
    .fin_divi:
    ret
```

5. Implementar un procedimiento que muestre los números pares desde el 0 al 100, utilizando la instrucción cmp y el salto condicional je.

```
    ; ------------- 4 - Para numeros pares -------------
    mov     edx,        msj4        ; Mostrar mensaje
    call    puts
    mov     dl,         0
    mov     cl,         2
    mov     dh,         100
    call    numeros
    call    salto
```

```
numeros:
    ;   Entrada:    dl -> Inicio
    ;               cl -> Incrementos
    ;               dh -> Fin
    ;   Utiliza:    eax
    ;   Salida:     Muesta los valores en consola
    .bucle:
        cmp     dh, dl
        jb      .fin_divi       ; Si es menor, terminamos
        mov     al,         dl
        call    printHex
        call    salto
        add     dl, cl
        jmp .bucle
    .fin_divi:
    ret
```

Instrucciones vistas en clase y permitidas para esta práctica.

1. add
2. sub
3. mov
4. jmp
5. cmp
6. je
7. loop

8. getch
9. getche
10. puts
11. putchar

6. Obligatoriamente de cada paso debe tener las capturas del código y de la terminal.

Conclusiones

La realización de esta práctica permitió comprender de manera más profunda el funcionamiento básico de la arquitectura i386 y la forma en que el procesador maneja los datos a través de sus registros. A diferencia de los lenguajes de alto nivel, el lenguaje ensamblador requiere un control directo sobre los registros, la memoria y el flujo de ejecución del programa, lo cual hace evidente cómo se ejecutan realmente las instrucciones dentro del procesador.

Uno de los aprendizajes más importantes fue entender la relación entre los registros principales y sus subregistros, como es el caso de EAX, AX, AH y AL, y cómo la modificación de uno de estos puede afectar el contenido del registro completo. Este conocimiento es fundamental para evitar sobrescribir datos accidentalmente y para manipular la información de forma correcta.

Dificultades

Durante el desarrollo de la práctica se presentaron algunas dificultades relacionadas con la comprensión del funcionamiento de los registros en la arquitectura i386. En particular, resultó complejo entender cómo se relacionan los subregistros con los registros principales. Por ejemplo, el registro AL forma parte del registro EAX [1] , por lo que al modificar el contenido de AL también se está alterando una parte del valor almacenado en EAX. Esto puede provocar errores si no se tiene cuidado al manipular los registros, ya que se pueden sobrescribir datos que posteriormente se necesitan.

Otra dificultad fue comprender el funcionamiento de las instrucciones de salto condicional (jump)[2]. Estas instrucciones generalmente dependen del resultado de una comparación realizada previamente con la instrucción (cmp). Dependiendo del resultado de esta comparación, el flujo del programa puede saltar hacia una sección posterior o regresar a una sección anterior del código. Entender cuándo utilizar cada tipo de salto (por ejemplo: JE, JNE, JG, JL, JGE, JLE) fue importante para implementar correctamente estructuras de control similares a condicionales y ciclos presentes en otros lenguajes de programación.

También resultó complicado comprender el funcionamiento de la rutina printHex utilizada para mostrar valores en formato hexadecimal. Esta rutina requiere que se le proporcione una cadena de al menos 8 bytes donde se almacenará el resultado de la conversión. Además, es necesario cargar previamente en el registro EAX el número que se desea convertir a hexadecimal. Si estos requisitos no se cumplen correctamente, la rutina no produce el resultado esperado.

A pesar de estas dificultades iniciales, la práctica permitió comprender mejor la manipulación directa de registros, el control del flujo del programa mediante instrucciones de salto y el uso de rutinas auxiliares para la visualización de datos.

Bibliografía

[1] D. A. Alpern, “El microprocesador 80386.” https://www.alpertron.com.ar/80386.HTM
[2] “Guide to x86 Assembly.” https://www.cs.virginia.edu/~evans/cs216/guides/x86.html